



國立清華大學
NATIONAL TSING HUA UNIVERSITY

Department of Computer Science and Information Engineering

IIS5008 Hardware Security Program Assignment 2: Hardware Trojan Insertion

黃偉祥 X1136010

Lecturer: Andy, Yu-Guang Chen

A report of IIS5008 course Program Assignment2
Institute of Information Security

May 2, 2025

Declaration

I, Wei-Xiang Wong, of the Department of Computer Science and Information Engineering, National Tsing Hua University, confirm that this is my own work, figures, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. I understand that failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalized accordingly.

I give my consent to having a copy of my report shared with future students as an example.

Wei-Xiang Wong May 2, 2025

Contents

1	About	1
1.1	Compilation	1
1.2	Tasks	1
2	My Trojan	2
2.1	Trigger	2
2.2	Payload	3
2.3	Trojan code	3
3	Wave & Explanation	5
3.1	AES	5
3.2	Sample HT	6
3.3	Reference HT	6
3.4	My HT	7
4	Testbench Explanation	9
4.1	AES	9
4.2	Sample HT	10
4.3	Reference paper HT	11
4.4	My Trojan	13
	References	16

1 About

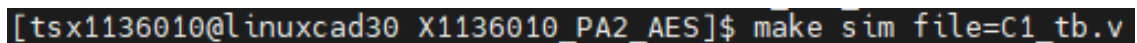
1.1 Compilation

Use **command 1** to compile and simulate the waveform, then use **command 2** to view the waveform.

Commands:

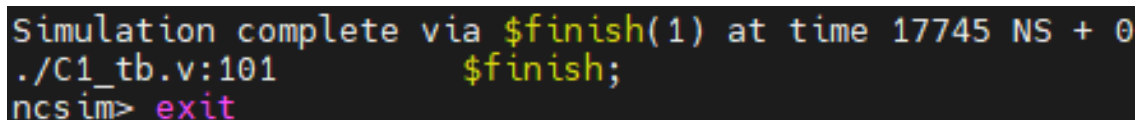
```
1 make sim file=C1_tb.v
2 make view file=AES_top_tb.fsdb
```

Listing 1.1: Command Line Usage



```
[tsx1136010@linuxcad30 X1136010_PA2_AES]$ make sim file=C1_tb.v
```

Figure 1.1: compile AES using make



```
Simulation complete via $finish(1) at time 17745 NS + 0
./C1_tb.v:101      $finish;
ncsim> exit
```

Figure 1.2: successful compilation

If compilation is successful, we will see **Figure 1.2** and also will generate file **AES_top_tb.fsdb**. After that, use **command 2** to view the wave form. Since I complete **all tasks** including sample HT, reference paper HT, and bonus HT, it should be tested in all folders as **1.2 tasks**.

1.2 Tasks

Under folders:

- X1136010_PA2_AES
- X1136010_PA2_sample_HT
- X1136010_PA2_reference_HT
- X1136010_PA2_bonus

2 My Trojan

My hardware Trojan combine **sequential** hardware Trojan and **combination** hardware Trojan, showed as **Figure 2.1**.

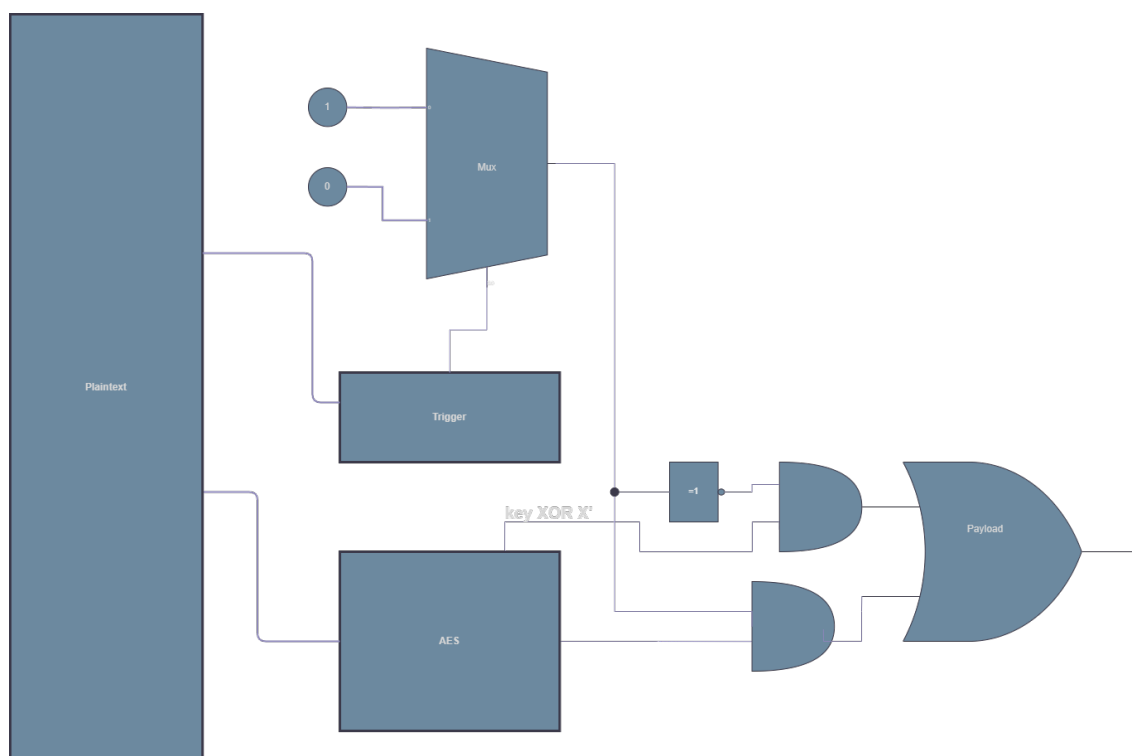


Figure 2.1: My design of Trojan

2.1 Trigger

Trojan will start working when several conditions meet at the same time

1. Specific input X when reach Q state
2. EN is made to logic 1 consecutively for Q times

$EN = 1$ only if $plaintext[a] == plaintext[b]$, $plaintext[a]$ is part of plaintext. In the other word $EN = 1$ only when some input pattern meet. This trojan will leak the $key \oplus \bar{X}$ for few cycles when all the conditions meet (triggered), and then act as normal and will still provide the correct **ciphertext** as usual.

key can be retrieved using $key \oplus \bar{X} \oplus \bar{X} == key$. Let the **payload** output $key \oplus \bar{X}$ can make the output have no information about key , it only can be retrieved by using the specific

X. This Trojan will be activated in very rare conditions meet, and for only few cycles. It make that an adversary very hard to detect the trigger condition.

2.2 Payload

Output is defined as $\text{key} \oplus \bar{X} * \bar{P} + \text{ciphertext} * P$

When $\text{trigger} == 1$, then $P = 1$ else $P = 0$, so when the Trojan is not activated then the output of AES will be ciphertext, and when the Trojan is activated then the output of AES will be $\text{key} \oplus \bar{X}$ for few cycles, then set trigger back to 0 and the output will become normal ciphertext.

2.3 Trojan code

```

1 assign out = (tmp & payload) | ((key^128'
   h1234_5678_abcd_dcba_abcd_cdab_4321_3412) & {128{~|payload}});
2 always @(trigger) begin
3   payload = 128'b0;
4 end
5 always @(~trigger) begin
6   payload = ~128'b0;
7 end
8 always @ (posedge clk) begin
9   s0 <= state ^ key;
10  k0 <= key;
11  if (previous_state!=state) begin
12    if(state[127:120] == state[7:0] && state[87:80] == state[47:40])
13      begin
14        ctr = ctr + 1;
15        if (ctr == 3 && state==128'
16          h1234_5678_abcd_dcba_abcd_cdab_4321_3412) begin
17          trigger <= 1;
18          trigger_delay <= 0;
19        end
20      end else begin
21        ctr = 0;
22        trigger <= 0;
23      end
24    end
25    previous_state <= state;
26 end
27 always @(posedge clk) begin
28   if (trigger == 1) begin
29     payload <= 128'b0;
30     trigger_delay <= trigger_delay + 1;
31   end
32   if (trigger_delay >= 100)begin
33     trigger <= 0;
34     trigger_delay <= 0;
35     ctr <= 0;
36   end
37 end

```

Listing 2.1: MyTrojan

In this example, the Trojan will leak $\text{key} \oplus \bar{X}$ for **100** cycles, **EN == 1** defined as when $\text{state}[127:120] == \text{state}[7:0]$ and $\text{state}[87:80] == \text{state}[47:40]$, **Q state** is once EN is made to logic 1 consecutively for 3 times, **X** is defined as **0x12345678abcdcbaabcdcdab43213412**.

This design make the Trojan can only been trigger is and only if the well known of the specific input and also the well knowledge of the circuit. It can evade manufacturing or production test patterns (e.g., stuck-at fault tests, and delay tests).

3 Wave & Explanation

3.1 AES

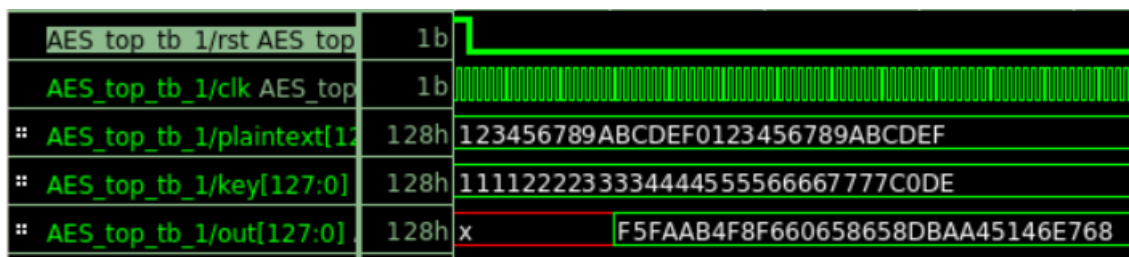


Figure 3.1: AES output wave

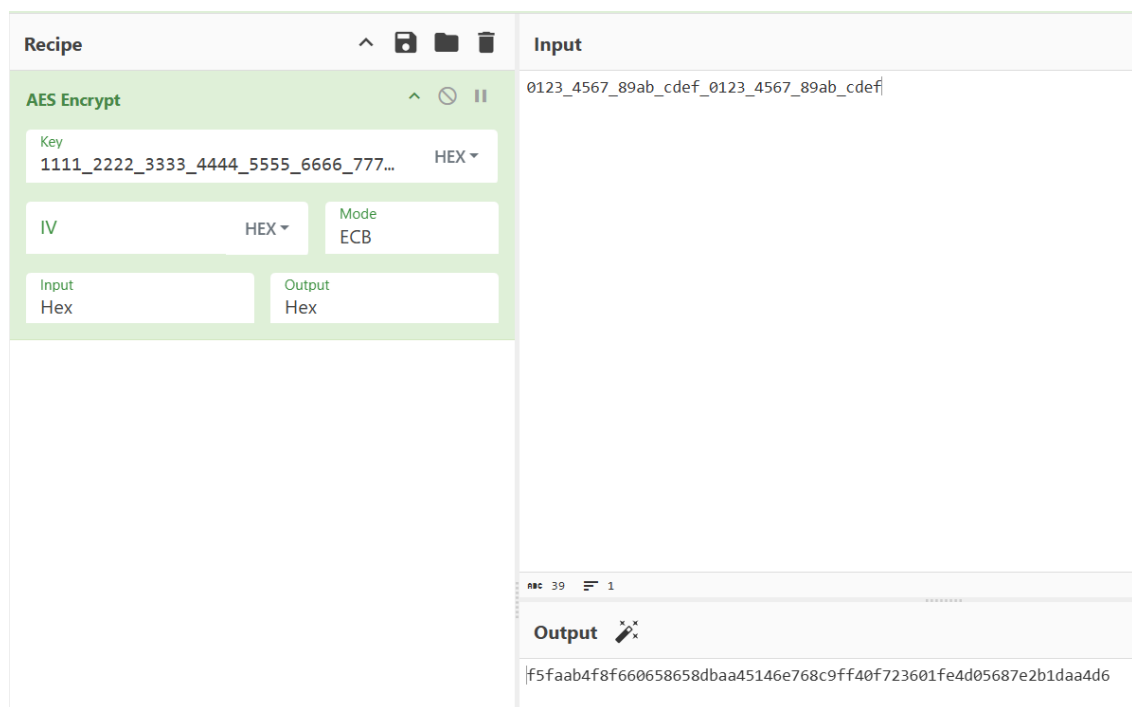


Figure 3.2: AES proved

The output(ciphertext) is **0xf5faab4f8f660658658dbaa45146e768**, when key = **0x1111222233334444555566667777C0DE**, input(plaintext) = **0x0123456789abcdef0123456789abcdef**. The output is ready after 19 cycles when rst = 0, and it is proved correct by using online AES encryption tool [CyberChef](#)

3.2 Sample HT

A sample hardware trojan, the cycle count start to count after the **rst == 0** at every **posedge clk**

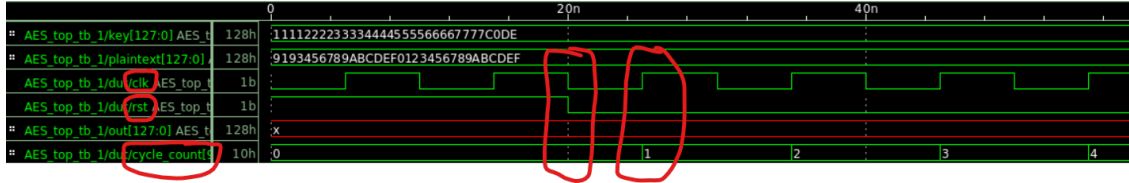


Figure 3.3: cycle count start when rst=0 at posedge clk

The key will be leak by this Trojan when the last 16 bits of the key equal 0xC0DE and the encryption process reaches 200(0xC8) clock cycles.

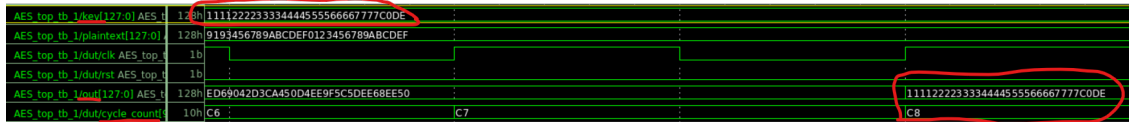


Figure 3.4: key leak when conditions meet

Output will back to normal ciphertext if the clock cycles is not 200.

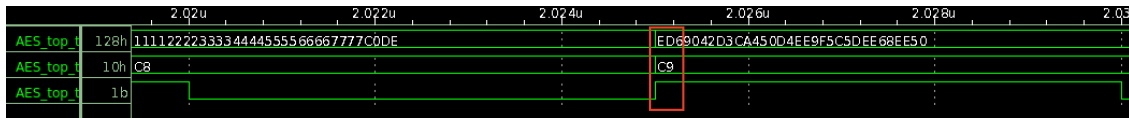


Figure 3.5: Return to ciphertext if not 200 clk cycles

The output will remain ciphertext even if cycle count reaches 200(0x200) if the key pattern is not match(last 16 bits of key not equal 0xC0DE).

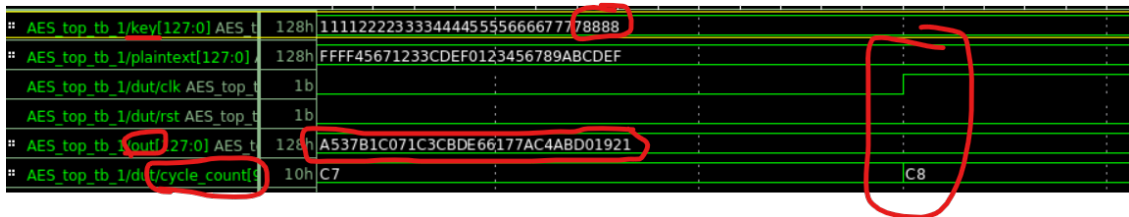


Figure 3.6: key no match pattern will no leak key

3.3 Reference HT

I use **127 bit** and **119 bit** as **EN** (en = state[127] & state[119] ;), EN will remain high if the input(plaintext) 127 and 119 bits are both 1, and **ctr** (state) will increasing until reaches **Q state**, ctr increasing (state update) only when new input.

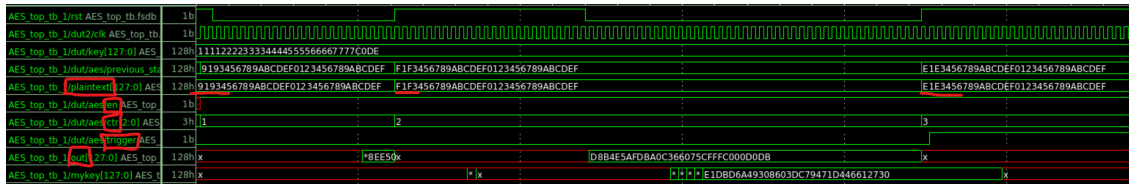


Figure 3.7: ctr as "Q state"

The round key K_{10} information will be leak to output when the Trojan is activated. In **Figure 3.7** example, I set the Q state as consecutively EN == 1 for 3 times, so when ctr == 3 then the output will be $K_{10} \oplus Y_2$, which Y_2 is **0x1616...1616**. Then we can reverse the key from K_{10} using the reverse algorithm that I implemented as **Reverse_key** module in **reverse.v** under the same folder. In Figure 3.7, **mykey** is the reversed key from output(ciphertext) using the reverse algorithm.

Note: Reverse algorithm take 15 clock cycles to reverse K_{10} to key

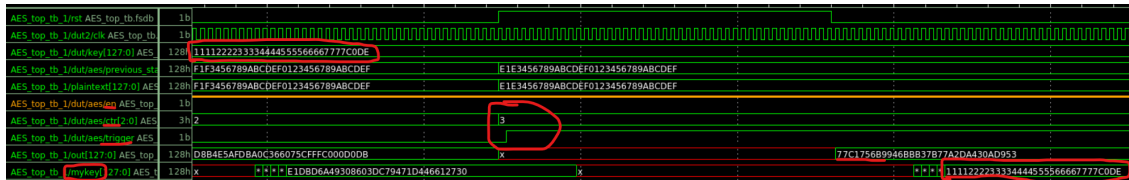


Figure 3.8: Reversed key when Q state

In **Figure 3.8**, circle with **orange** color is when reached Q state, while circle with **red** color are when the FSM back to initial state S_0 .

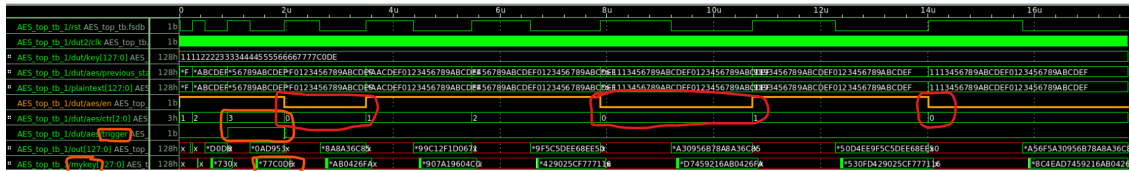


Figure 3.9: FSM state transformation

3.4 My HT

Int **Figure 3.9**, if the Trojan been triggered will only been activated for 100 cycles and after that the AES will work as usual, **my_out** is the key that **output** $\oplus \tilde{X}$. In this example, the Trojan will leak **key** $\oplus \tilde{X}$ for **100** cycles, **EN == 1** defined as when plaintext[127:120] == plaintext[7:0] and plaintext[87:80] == plaintext[47:40], **Q state** is once EN is made to logic 1 consecutively for 3 times, **X** is defined as **0x12345678abcdcbaabcdcdab43213412**

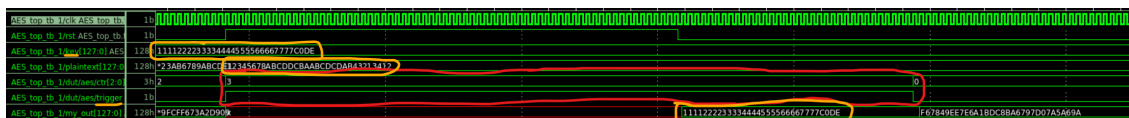


Figure 3.10: Leak key information when reaches Q-state and specific input pattern

In **Figure 3.10**, in this case the key information will **not** be leak even reached **Q-state** because the input pattern at Q-state is not match. The FSM is similar to [Jain and Guin \(2020\)](#), but the state can keep going to more than Q-state.

Note: Trojan will only be triggered at Q-state if input matched for 100 cycles.

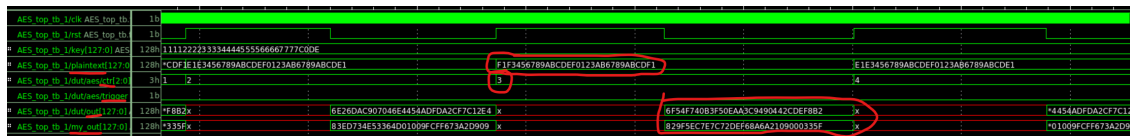


Figure 3.11: Enter Caption

4 Testbench Explanation

4.1 AES

```
ncsim> run
rst=1, key=c0de, out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
rst=0, key=c0de, out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
rst=0, key=c0de, out=f5faab4f8f660658658dbaa45146e768
```

Figure 4.1: AES result on screen

In this testbench, **rst** is set to **1**, **plaintext** is **0x0123456789abcdef0123456789abcdef**, and **key** = **0x1111222233334444555566667777CODE**, then **rst** is set to 0 at **20ns**. This **simulation** will end at **4000ns**, and it will print information when **rst**, **key**, or **out** changed.

```
1 // Dump waveforms for debugging
2 initial begin
3     $dumpfile("AES_top_tb.fsdb"); // Waveform output file
4     $dumpvars(0, AES_top_tb_1);   // Dump all signals in this
    testbench
5 end
6 // Generate a clock with 10 ns period (toggle every 5 ns)
7 initial begin
8     clk = 0;
9     forever #5 clk = ~clk;
10 end
11 // Assign test conditions
12 initial begin
13     // Initially assert reset and set default inputs
14     rst = 1;
15     plaintext = 128'h0123_4567_89ab_cdef_0123_4567_89ab_cdef;
16     key = 128'h1111_2222_3333_4444_5555_6666_7777_CODE;
17     #20;
18     rst = 0;
19
20     #4000;
21     // End simulation
22     $finish;
23 end
24 // Monitor signals
25 initial begin
26     $monitor("rst=%b, key=%h, out=%h\n",
27             rst, key[15:0], out);
```

```
28 end
```

Listing 4.1: AES testbench

4.2 Sample HT

```
ncsim> run
cycle_count= 200, key=c0de, out=ed69042d3ca450d4ee9f5c5dee68ee50
cycle_count= 200, key=c0de, out=1111222233334444555566667777c0de
cycle_count= 200, key=8888, out=a537b1c071c3cbde66177ac4abd01921
```

Figure 4.2: output of sample HT on screen

This testbench have 2 different plaintext and key, but this time I only show when clock cycles reached 200, the first result will leak the key when condition meet, if key pattern is not match then the key will not leak even clock cycles reached 200.

Note: First output is the original output before clock reach 200.

```
1 // Apply stimulus
2 initial begin
3     // Initially assert reset and set default inputs
4     rst = 1;
5     plaintext = 128'h9193_4567_89ab_cdef_0123_4567_89ab_cdef;
6     key = 128'h1111_2222_3333_4444_5555_6666_7777_CODE;
7     // plaintext2 = 128'h9193_4567_89ab_cdef_0123_4567_89ab_cdef;
8     #20;
9     rst = 0;
10
11     #4000;
12     rst=1;
13     plaintext = 128'hffff_4567_1233_cdef_0123_4567_89ab_cdef;
14     key = 128'h1111_2222_3333_4444_5555_6666_7777_8888;
15
16     #4020;
17     rst=0;
18
19
20     #8000;
21
22     // End simulation
23     $finish;
24 end
25
26 // Monitor signals
27 always @(*) begin
28     if(dut.cycle_count == 200)begin
29         $display("cycle_count=%d, key=%h, out=%h\n", dut.cycle_count, key
30         [15:0], out);
31     end
32 end
```

Listing 4.2: Sample HT testbench

4.3 Reference paper HT

```

ncsim> run
rst=0, en=1, key=c0de, out=ed69042d3ca450d4ee9f5c5dee68ee50, ctr=1
rst=1, en=1, key=c0de, out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx, ctr=2
rst=0, en=1, key=c0de, out=0d8b4e5afdba0c366075cfff000d0db, ctr=2
rst=1, en=1, key=c0de, out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx, ctr=3
Trigger:1, K10 = 61d7637d8f50ada56d61b4cc551ccf45, reversed_key=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
rst=0, en=1, key=c0de, out=77c1756b9946bbb37b77a2da430ad953, ctr=3
Trigger:1, K10 = 61d7637d8f50ada56d61b4cc551ccf45, reversed_key=1111222233334444555566667777c0de
rst=1, en=0, key=c0de, out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx, ctr=0
rst=0, en=0, key=c0de, out=8b57703971a56f5a30956b78a8a36c85, ctr=0
rst=1, en=1, key=c0de, out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx, ctr=1
rst=0, en=1, key=c0de, out=5e41542f80d822fbaa5c99c12f1d0671, ctr=1
rst=1, en=1, key=c0de, out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx, ctr=2
rst=0, en=1, key=c0de, out=ed69042d3ca450d4ee9f5c5dee68ee50, ctr=2
rst=1, en=0, key=c0de, out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx, ctr=0
rst=0, en=0, key=c0de, out=8b57703971a56f5a30956b78a8a36c85, ctr=0
rst=1, en=1, key=c0de, out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx, ctr=1
rst=0, en=1, key=c0de, out=ed69042d3ca450d4ee9f5c5dee68ee50, ctr=1
rst=1, en=0, key=c0de, out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx, ctr=0
rst=0, en=0, key=c0de, out=8b57703971a56f5a30956b78a8a36c85, ctr=0

```

Figure 4.3: Reference HT testbench result on screen

This testbench have severals plaintext with a unique key, it will print the reversed key using reverse algorithm. If the condition is not meet (EN==0) then state will return to initial state.

```

1 initial begin
2     // Initially assert reset and set default inputs
3     rst = 1;
4     plaintext = 128'h9193_4567_89ab_cdef_0123_4567_89ab_cdef;
5     key = 128'h1111_2222_3333_4444_5555_6666_7777_CODE;
6     #20;
7     rst = 0;
8
9     #225;
10    rst = 1;
11    plaintext = 128'hf1f3_4567_89ab_cdef_0123_4567_89ab_cdef;
12    #235;
13    rst=0;
14
15    #415;
16    rst = 1;
17    plaintext = 128'he1e3_4567_89ab_cdef_0123_4567_89ab_cdef;
18    #425;
19    rst=0;
20
21    #645;
22    rst=1;
23    plaintext = 128'h1113_4567_89ab_cdef_0123_4567_89ab_cdef;
24    #665;

```

```

25     rst=0;
26
27     #865;
28     rst=1;
29     plaintext = 128'hf1f3_4567_89aa_cdef_0123_4567_89ab_cdef;
30     #885;
31     rst=0;
32
33     #1085;
34     rst = 1;
35     plaintext = 128'h9193_4567_89ab_cdef_0123_4567_89ab_cdef;
36     #1105;
37     rst=0;
38
39     #1305;
40     rst=1;
41     plaintext = 128'h1113_4567_89ab_cdef_0123_4567_89ab_cdef;
42     #1325;
43     rst=0;
44
45     #1525;
46     rst = 1;
47     plaintext = 128'h9193_4567_89ab_cdef_0123_4567_89ab_cdef;
48     #1545;
49     rst=0;
50
51     #1745;
52     rst=1;
53     plaintext = 128'h1113_4567_89ab_cdef_0123_4567_89ab_cdef;
54     #1765;
55     rst=0;
56
57     #1965;
58     // End simulation
59     $finish;
60 end
61
62 always @(out)begin
63
64     if (dut.aes.trigger==1) begin
65         $display("Trigger=%d, K10 = %h, reversed_key=%h",dut.aes.trigger ,
66         dut.aes.k9b,dut2.key);
67     end
68     $display("rst=%b, en=%b, key=%h, out=%h, ctr=%d\n",rst,dut.aes.en,key
69     [15:0],out,dut.aes.ctr);
70 end

```

Listing 4.3: Reference paper HT testbench

4.4 My Trojan

```

ncsim> run
ctr=3, ciphertext=6e26dac907046e4454adfdac2cf7c12e4
trigger=0, my_out=83ed734e53364d01009fcff673a2d909

ctr=3, ciphertext=6e26dac907046e4454adfdac2cf7c12e4
trigger=1, my_out=83ed734e53364d01009fcff673a2d909

ctr=3, ciphertext=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
trigger=1, my_out=83ed734e53364d01009fcff673a2d909

ctr=3, ciphertext=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
trigger=1, my_out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

ctr=3, ciphertext=fcda8ba56701670101675432cba90b33
trigger=1, my_out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

ctr=3, ciphertext=fcda8ba56701670101675432cba90b33
trigger=1, my_out=1111222233334444555566667777c0de

ctr=3, ciphertext=6e26dac907046e4454adfdac2cf7c12e4
trigger=0 my_out=83ed734e53364d01009fcff673a2d909

ctr=3, ciphertext=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
trigger=0 my_out=83ed734e53364d01009fcff673a2d909

ctr=3, ciphertext=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
trigger=0 my_out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

ctr=3, ciphertext=6f54f740b3f50eaa3c9490442cdef8b2
trigger=0 my_out=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

ctr=3, ciphertext=6f54f740b3f50eaa3c9490442cdef8b2
trigger=0 my_out=829f5ec7e7c72def68a6a2109000335f

```

Figure 4.4: My Trojan result on screen

In this testbench also have severals plaintext and a unique key, but this time it only show on screen when reached Q-state, and the Trojan will only been activated if reached Q-state and input pattern matched.

Note: Trojan only leak the key information for 100 cycles.


```
1 // Apply stimulus
2 initial begin
3     // Initially assert reset and set default inputs
4     rst = 1;
5     plaintext = 128'h9193_4567_89ab_cdef_0123_4567_89ab_cdef;
6     key = 128'h1111_2222_3333_4444_5555_6666_7777_CODE;
7     #20;
8     rst = 0;
9
10    #205;
11    rst = 1;
12    plaintext = 128'hf1f3_4567_89ab_cdef_0123_ab67_89ab_cdf1;
13    #225;
14    rst=0;
15
16    #425;
17    rst = 1;
18    plaintext = 128'he1e3_4567_89ab_cdef_0123_ab67_89ab_cde1;
19    #445;
20    rst=0;
21
22    #645;
23    rst=1;
24    plaintext = 128'h1234_5678_abcd_dcba_abcd_cdab_4321_3412;
25    #665;
26    rst=0;
27
28    #865;
29    rst=1;
30    plaintext = 128'h9193_4567_89ab_cdef_0123_4567_89ab_cdef;
31    #885;
32    rst=0;
33
34    #1085;
35    rst = 1;
36    plaintext = 128'hf1f3_4567_89ab_cdef_0123_ab67_89ab_cdf1;
37    #1105;
38    rst=0;
39
40    #1305;
41    rst=1;
42    plaintext = 128'he1e3_4567_89ab_cdef_0123_ab67_89ab_cde1;
43    #1325;
44    rst=0;
45
46    #1525;
47    rst = 1;
48    plaintext = 128'hf1f3_4567_89ab_cdef_0123_ab67_89ab_cdf1;
49    #1545;
50    rst=0;
51
52    #1745;
53    rst=1;
54    plaintext = 128'he1e3_4567_89ab_cdef_0123_ab67_89ab_cde1;
55    #1765;
56    rst=0;
57
58    #1965;
59
60    // End simulation
```

```
61     $finish;
62 end
63 always @(*) begin
64     if(dut.aes.ctr == 3) begin
65         $display("ctr=%d, ciphertext=%h\ntrigger=%d, my_out=%h\n",dut.aes.
66         ctr,out,dut.aes.trigger,my_out);
67     end
68 end
```

Listing 4.4: My HT testbench

References

Jain, A. and Guin, U. (2020), A novel tampering attack on aes cores with hardware trojans, *in* '2020 IEEE International Test Conference in Asia (ITC-Asia)', pp. 77–82.